Forensic Evidence & Chain of Custody System

Final Project Report

Project Team:

K S Nithurshen	– 2410110157
A Sanjith	– 2410110001
Tanish Shitanshu	– 2510110008
Mohammad Ibrahim Zaki	– 2410110205

Project Repository:

https://github.com/Nithurshen/forensic-coc-system

Contents

1 Project Overview **2**
1.1 Core Features 2

2 Database Schema Diagram **3**
2.1 Relational Model Mapping 3

3 Cryptographic Row-Linking **4**

4 Detailed Code Analysis **5**
4.1 Database Initialization (`init_db.py`) 5
4.2 Database Management (`db_manager.py`) 5
4.3 The Cryptographic Ledger (`crypto_ledger.py`) 6

5 Core SQL Commands (DML & DDL) **7**

6 Applied Database Management System (DBMS) Concepts **8**
6.1 Referential Integrity (Primary & Foreign Keys) 8
6.2 Many-to-Many Relationships (Junction Tables) 8
6.3 Relational Joins (INNER JOIN) 8
6.4 ACID Properties: Atomicity (Transactions) 9
6.5 Database Cursors 9
6.6 Database Triggers 10

7 Stored Procedures and Stored Functions **11**
7.1 Stored Procedure: Logging Temperature Alerts 11
7.2 Stored Function: Fetching Current Location 11

8 Functional Dependencies and Normalization **12**
8.1 Functional Dependencies (FDs) 12
8.2 Normalization Evaluation 12

9 Limitations **13**

10 References and URLs **14**

1 Project Overview

The **Forensic Evidence & Chain of Custody System** (`forensic-coc-system`) is a high-security database architecture designed specifically for crime laboratories and police departments. Its primary purpose is to manage the collection, analysis, storage, and court presentation of forensic evidence.

Unlike standard inventory management applications that simply update the state of an item (e.g., changing its location from "Lab" to "Storage"), this system replaces state changes with strict **transfers of custody**. Evidence records are immutable, prioritizing absolute data integrity to ensure strict legal and court admissibility.

1.1 Core Features

- **Cryptographic Ledger:** Replaces standard state changes with strict, mathematically verifiable transfers of custody using SHA-256 hashing.
- **Interactive Streamlit Frontend:** A fully functional GUI providing role-based authentication, real-time metrics dashboards, and interactive forms.
- **Digital Fingerprinting:** Allows investigators to upload digital evidence (e.g., CCTV footage), generating an immediate cryptographic hash to track digital alteration.
- **Automated PDF Reporting:** Integrates Python `fpdf` to dynamically query the chain of custody and compile court-ready documents.
- **Environmental Monitoring:** Tracks sensitive physical evidence and alerts officers if storage temperatures breach safe thresholds.

2 Database Schema Diagram

The following entity-relationship diagram illustrates the complete relational structure of the forensic chain of custody system, including primary keys, foreign keys, and table relationships.

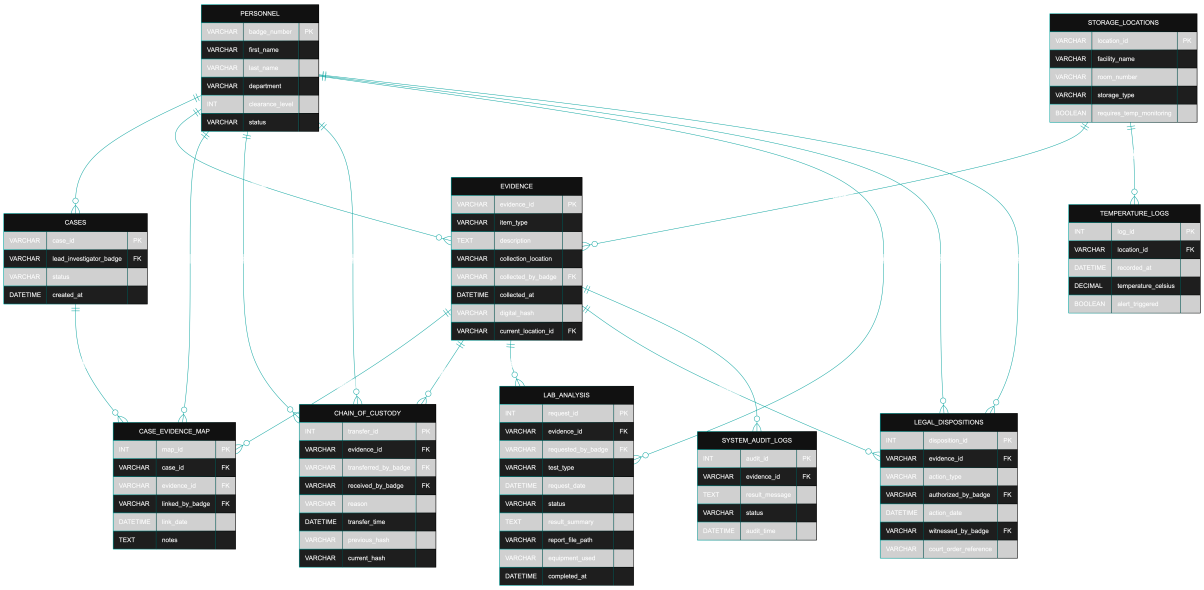

Figure 1: Entity-Relationship Diagram of the Forensic CoC System

2.1 Relational Model Mapping

The ER Diagram maps to the following normalized relational schema (Primary Keys are <u>underlined</u>, Foreign Keys are *italicized*):

- **personnel** (<u>badge_number</u>, username, password_hash, first_name, last_name, department, clearance_level, status)
- **storage_locations** (<u>location_id</u>, facility_name, room_number, storage_type, requires_temp_monito
- **cases** (<u>case_id</u>, *lead_investigator_badge*, status, created_at)
- **evidence** (<u>evidence_id</u>, item_type, description, collection_location, *collected_by_badge*, collected_at, digital_hash, *current_location_id*)
- **case_evidence_map** (<u>map_id</u>, *case_id*, *evidence_id*, *linked_by_badge*, link_date, notes)
- **chain_of_custody** (<u>transfer_id</u>, *evidence_id*, *transferred_by_badge*, *received_by_badge*, reason, transfer_time, previous_hash, current_hash)
- **lab_analysis** (<u>request_id</u>, *evidence_id*, *requested_by_badge*, test_type, request_date, status, result_summary, report_file_path, equipment_used, completed_at)
- **temperature_logs** (<u>log_id</u>, *location_id*, recorded_at, temperature_celsius, alert_triggered)
- **legal_dispositions** (<u>disposition_id</u>, *evidence_id*, action_type, *authorized_by_badge*, action_date, *witnessed_by_badge*, court_order_reference)

3 Cryptographic Row-Linking

Standard SQL audit tables are vulnerable to manipulation by actors with high-level database access. The core innovation of this system is the implementation of a **Tamper-Evident SQL Ledger** directly within a relational database, avoiding the computational overhead and latency of external blockchain infrastructure.

Every transfer of custody generates a `current_hash` alongside a `previous_hash`. When a new transfer occurs, the system calculates a SHA-256 hash combining the new transaction payload with the hash of the preceding transaction for that specific item. If any historical record is altered directly in the database, the mathematical chain is instantly invalidated, providing cryptographic proof of database integrity.

4 Detailed Code Analysis

The backend logic, database schema, and cryptographic engine have been fully implemented. Below is an intricate breakdown of every completed module.

4.1 Database Initialization (`init_db.py`)

This script establishes the foundational relational schema in MySQL. It defines the entities required for a forensic environment.

- **Connection & Creation:** It connects via `mysql.connector` using credentials from a `.env` file and creates the database if it doesn't exist.
- **Core Tables:**
  - `personnel`: Stores actors (officers, lab techs) with their badge numbers and clearance levels.
  - `storage_locations`: Defines physical locations (e.g., Deep Freezer) and flags if they require temperature monitoring.
  - `evidence`: The central entity. It stores metadata like item type, collection location, and a `digital_hash` for digital evidence.
  - `chain_of_custody`: The ledger table. It contains `transferred_by_badge`, `received_by_badge`, `previous_hash`, and `current_hash`.
  - `lab_analysis` & `legal_dispositions`: Tracks requests for testing and court-ordered actions.

4.2 Database Management (`db_manager.py`)

This module acts as the Data Access Object (DAO), providing modular functions to interact with the database.

```python
def get_latest_hash(evidence_id):
    # Fetches the current_hash of the most recent transfer
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute(
        "SELECT current_hash FROM chain_of_custody WHERE evidence_id = %s ORDER BY transfer_id DESC LIMIT 1",
        (evidence_id,)
    )
    result = cursor.fetchone()
    return result["current_hash"] if result else "GENESIS_HASH"
```

Listing 1: Extracting the Previous Hash

Explanation: Before a new transfer can be logged, the system must know the cryptographic state of the previous transfer. If this is the first transfer, it defaults to a `GENESIS_HASH`.

4.3 The Cryptographic Ledger (crypto_ledger.py)

This is the central nervous system of the novelty. It ensures mathematical immutability.

```python
def process_new_transfer(transfer_payload):
    evidence_id = transfer_payload["evidence_id"]
    previous_hash = db_manager.get_latest_hash(evidence_id)
    transfer_payload["previous_hash"] = previous_hash

    current_hash = generate_row_hash(transfer_payload)
    transfer_payload["current_hash"] = current_hash

    inserted_id = db_manager.insert_transfer(transfer_payload)
    return True if inserted_id else False
```

Listing 2: Processing a New Transfer

Explanation: This orchestrates the transfer. It injects the `previous_hash` into the payload, generates the new `current_hash`, and signals the database manager to insert the record.

5 Core SQL Commands (DML & DDL)

The system heavily executes dynamic SQL commands through the backend. Below are key examples of the queries utilized.

```sql
-- Fetching full Chain of Custody with human-readable officer names
SELECT c.transfer_time, c.reason,
       p1.last_name AS transferred_by_name,
       p2.last_name AS received_by_name,
       c.current_hash
FROM chain_of_custody c
JOIN personnel p1 ON c.transferred_by_badge = p1.badge_number
JOIN personnel p2 ON c.received_by_badge = p2.badge_number
WHERE c.evidence_id = %s
ORDER BY c.transfer_time ASC;
```

Listing 3: Complex Relational Join for Court Audits

```sql
-- Identifying evidence in insecure storage or failing audits
SELECT e.evidence_id, e.description, l.storage_type
FROM evidence e
JOIN storage_locations l ON e.current_location_id = l.location_id
WHERE l.storage_type LIKE '%Temporary%'
OR e.evidence_id IN (
    SELECT evidence_id FROM system_audit_logs WHERE status = 'Fail'
);
```

Listing 4: Subqueries and Pattern Matching for Security Alerts

6 Applied Database Management System (DBMS) Concepts

The project heavily relies on fundamental and advanced DBMS concepts to guarantee data integrity, relational mapping, and efficient querying. Below is an intricate breakdown of the concepts utilized and exactly where they are implemented in the codebase.

6.1 Referential Integrity (Primary & Foreign Keys)

Concept: Ensuring that relationships between tables remain consistent. A foreign key in one table must match a valid primary key in another.
Implementation: Extensively used in `init_db.py`.

```sql
CREATE TABLE IF NOT EXISTS evidence (
    evidence_id VARCHAR(255) PRIMARY KEY,
    collected_by_badge VARCHAR(50) NOT NULL,
    current_location_id VARCHAR(100) NOT NULL,
    FOREIGN KEY (collected_by_badge) REFERENCES personnel(badge_number),
    FOREIGN KEY (current_location_id) REFERENCES storage_locations(location_id)
)
```

Listing 5: Enforcing relationships in the Evidence Table

How it's utilized: This prevents the creation of "orphan" records. You cannot log a piece of evidence if the officer's badge number doesn't exist in the `personnel` table, or if the storage location doesn't exist in the `storage_locations` table.

6.2 Many-to-Many Relationships (Junction Tables)

Concept: A relational design pattern used when multiple records in Table A can relate to multiple records in Table B. This is resolved by creating an intermediate "Junction Table."
Implementation: The `case_evidence_map` table in `init_db.py`.

```sql
CREATE TABLE IF NOT EXISTS case_evidence_map (
    map_id INT AUTO_INCREMENT PRIMARY KEY,
    case_id VARCHAR(255) NOT NULL,
    evidence_id VARCHAR(255) NOT NULL,
    FOREIGN KEY (case_id) REFERENCES cases(case_id),
    FOREIGN KEY (evidence_id) REFERENCES evidence(evidence_id)
)
```

Listing 6: Junction Table for Cases and Evidence

How it's utilized: A single case might have multiple pieces of evidence. Conversely, a single piece of evidence might be linked to multiple different cases over time. The `case_evidence_map` allows this complex mapping without duplicating evidence data.

6.3 Relational Joins (INNER JOIN)

Concept: Combining rows from two or more tables based on a related column between them to present a unified view of the data.
Implementation: The `get_full_chain_of_custody` function in `db_manager.py`.

```
cursor.execute("""
    SELECT c.transfer_time, c.reason,
           p1.last_name AS transferred_by_name,
           p2.last_name AS received_by_name
    FROM chain_of_custody c
    JOIN personnel p1 ON c.transferred_by_badge = p1.badge_number
    JOIN personnel p2 ON c.received_by_badge = p2.badge_number
    WHERE c.evidence_id = %s
    ORDER BY c.transfer_time ASC
""", (evidence_id,))
```

Listing 7: Using JOINs for Court Reporting

How it's utilized: The `chain_of_custody` table only stores badge numbers to save space and maintain normalization. However, for a court report, human-readable names are required. This query uses two aliases (`p1` and `p2`) to join the `personnel` table twice to the same row, extracting both the transferring officer's name and the receiving officer's name.

6.4 ACID Properties: Atomicity (Transactions)

Concept: Ensuring that a database operation completely succeeds or completely fails, leaving no partial data behind.
Implementation: The explicit use of `conn.commit()` in all insert/update functions within `db_manager.py`. **How it's utilized:** If an `execute` command succeeds, `conn.commit()` permanently writes it to the database. If it fails, an exception is raised, the commit does not happen, and the database remains in its previous, uncorrupted state.

6.5 Database Cursors

Concept: A cursor is a control structure that enables traversal over the records in a database. It acts as an iterator, allowing the backend application to process individual rows returned by a query.
Implementation: Extensively implemented in `db_manager.py` and `init_db.py` using the `mysql.connector` library.

```
def get_full_chain_of_custody(evidence_id):
    conn = get_db_connection()
    # Instantiating a cursor that returns rows as Python dictionaries
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT ... FROM chain_of_custody WHERE evidence_id
   = %s", (evidence_id,))

    # Traversing the result set
    results = cursor.fetchall()
    conn.close()
    return results
```

Listing 8: Utilizing Dictionary Cursors for Data Traversal

How it's utilized: The system utilizes `cursor(dictionary=True)`, which maps the SQL column names to key-value pairs. This makes the data traversal highly efficient when passing the SQL payload directly to the JSON-dependent `generate_row_hash` function in the cryptographic ledger.

6.6 Database Triggers

The system utilizes triggers to automate audit logging when critical data is modified.

```sql
DELIMITER //
CREATE TRIGGER AfterCaseUpdate
AFTER UPDATE ON cases
FOR EACH ROW
BEGIN
    IF OLD.status != NEW.status THEN
        INSERT INTO system_audit_logs (evidence_id, result_message, status, audit_time)
        VALUES ('SYSTEM', CONCAT('Case ', NEW.case_id, ' status changed to ', NEW.status), 'Pass', NOW());
    END IF;
END; //
DELIMITER ;
```

Listing 9: Native SQL Trigger for Case Status Auditing

Furthermore, the concept of a trigger is heavily utilized at the **Application Level** within `crypto_ledger.py`, where Python intercepts the data transfer and enforces the cryptographic rule *before* calling the `INSERT` statement.

7 Stored Procedures and Stored Functions

To push processing logic closer to the data layer and reduce application-side overhead, the system utilizes native SQL Stored Procedures and Functions.

7.1 Stored Procedure: Logging Temperature Alerts

Certain forensic evidence (like DNA) requires strict temperature control. This stored procedure abstracts the logic of determining if an environmental alert should be triggered based on a sensor reading.

```sql
DELIMITER //
CREATE PROCEDURE LogTemperatureAlert(
    IN p_loc_id VARCHAR(100),
    IN p_temp DECIMAL(5,2)
)
BEGIN
    DECLARE v_alert_flag BOOLEAN;

    -- If freezer temperature rises above -10.0 C, trigger an alert
    IF p_temp > -10.0 THEN
        SET v_alert_flag = TRUE;
    ELSE
        SET v_alert_flag = FALSE;
    END IF;

    INSERT INTO temperature_logs (location_id, recorded_at,
   temperature_celsius, alert_triggered)
    VALUES (p_loc_id, NOW(), p_temp, v_alert_flag);
END //
DELIMITER ;
```

Listing 10: Stored Procedure for Environmental Monitoring

7.2 Stored Function: Fetching Current Location

This function provides a reusable, deterministic query to instantly locate physical evidence without writing repetitive `SELECT` statements.

```sql
DELIMITER //
CREATE FUNCTION GetCurrentLocation(p_ev_id VARCHAR(255))
RETURNS VARCHAR(100)
DETERMINISTIC
BEGIN
    DECLARE v_loc VARCHAR(100);

    SELECT current_location_id INTO v_loc
    FROM evidence
    WHERE evidence_id = p_ev_id;

    RETURN v_loc;
END //
DELIMITER ;
```

Listing 11: Stored Function for Evidence Tracking

8 Functional Dependencies and Normalization

A critical evaluation of the database schema was conducted to determine its highest normal form.

8.1 Functional Dependencies (FDs)

Below are the identified FDs for the primary tables:

1. **Table: personnel**
   - `badge_number` → `first_name, last_name, department, clearance_level, status`
2. **Table: storage_locations**
   - `location_id` → `facility_name, room_number, storage_type, requires_temp_monitoring`
3. **Table: evidence**
   - `evidence_id` → `item_type, description, collection_location, collected_by_badge, collected_at, digital_hash, current_location_id`
4. **Table: cases**
   - `case_id` → `lead_investigator_badge, status, created_at`

8.2 Normalization Evaluation

By evaluating the functional dependencies against normalization rules, the entire schema achieves **Boyce-Codd Normal Form (BCNF)**.

1. **1NF (First Normal Form):** All attributes contain atomic, indivisible values. There are no repeating groups or arrays within columns.
2. **2NF (Second Normal Form):** The tables are in 1NF, and there are no partial dependencies because all tables rely on a single-attribute primary key (e.g., `badge_number`, `evidence_id`).
3. **3NF (Third Normal Form):** The tables are in 2NF, and there are no transitive dependencies. Every non-prime attribute depends *only* on the primary key. For example, in the `evidence` table, the `description` depends entirely on the `evidence_id`, not on any other non-prime attribute.
4. **BCNF (Boyce-Codd Normal Form):** For every functional dependency $X \to Y$ identified above, the determinant (X) is a candidate key. Therefore, the schema is strictly in BCNF, ensuring zero data redundancy or update anomalies.

9 Limitations

While the system provides a robust framework, it currently operates with the following limitations:

- **Centralized Database:** The system relies on a centralized MySQL server. While the mathematical ledger detects tampering, a root user could theoretically drop the entire database table. A future enhancement would mirror the hashes to an actual decentralized blockchain.
- **Biometric Absence:** Authentication relies on standard hashed passwords (`bcrypt`). Future iterations should integrate biometric hardware APIs (fingerprint/RFID) for physical evidence hand-offs.
- **Storage Constraints:** Digital evidence hashing currently processes files directly in memory. For massive forensic disk images (1TB+), the system needs to implement chunked file-streaming to prevent memory overflow.

10 References and URLs

- Streamlit Documentation (Frontend Framework): https://docs.streamlit.io/
- MySQL Python Connector Official Guide: https://dev.mysql.com/doc/connector-python/en/
- Python hashlib (SHA-256 Cryptography): https://docs.python.org/3/library/hashlib.html
- FPDF Library (Automated Reporting): https://pyfpdf.readthedocs.io/
- Bcrypt Library (Password Hashing): https://pypi.org/project/bcrypt/